

Programa 1

Erick Jesús Ríos González

04/diciembre/2024

1. Problema de la mochila

1.1. Pseudocódigo del algoritmo

Algorithm 1 checkKnapsack Function

```
1: Function checkKnapsack( $C, O, w, v, k$ ):  
2: # Non-deterministic phase  
3:  $s \leftarrow \emptyset$  {Initialize empty set}  
4: for  $i \in O$  do  
5:   result  $\leftarrow$  flip a coin  
6:   if result = heads then  
7:      $s \leftarrow s \cup \{i\}$  {Add  $i$  to set  $s$ }  
8:   end if  
9: end for  
10: # Verification phase  
11:  $totalWeight \leftarrow 0$   
12:  $totalProfit \leftarrow 0$   
13: for  $i \in s$  do  
14:    $totalWeight \leftarrow totalWeight + w(i)$   
15: end for  
16: if  $totalWeight > C$  then  
17:   return 'NO'  
18: end if  
19: for  $i \in s$  do  
20:    $totalProfit \leftarrow totalProfit + v(i)$   
21: end for  
22: if  $totalProfit < k$  then  
23:   return 'NO'  
24: end if  
25: return 'YES'
```

1.2. Demostración de correctitud del algoritmo

El algoritmo no determinista checkKnapsack es correcto para el Problema de la Mochila.

Sea $C \in \mathbb{Z}^+$, la capacidad de la mochila. O , un conjunto de n objetos. $w : O \rightarrow \mathbb{Z}^+$, dado por $w(o_i) = x$, $x \in \{1, \dots, 100\}$ para cualquier $o_i \in O$.

$v : O \rightarrow \mathbb{Z}^+$, dado por $v(o_i) = y$, $y \in \{1, \dots, 100\}$ para cualquier $o_i \in O$.
 $k \in \mathbb{Z}^+$, $0 < k \leq \sum_{i=1}^n v(o_i)$.

Primero, supongamos que en la fase adivinadora se construye un subconjunto $S \subseteq O$ tal que el costo w de S no sobrepasa la capacidad C de la mochila y que la ganancia total v de S sea mayor o igual a k . Si esto sucede: En la fase verificadora no pasará al **if** de la línea 16 ya que no sobrepasa C . Al calcular la ganancia total, como será mayor o igual a k , no entrará en el **if** de la línea 22, por lo que llega hasta la línea 23 y regresa **YES**.

Ahora, supongamos que en la fase adivinadora se construye un subconjunto $S \subseteq O$ tal que el costo w de S sobrepasa la capacidad C . Por la línea 16, se ejecutará el **if** y regresará **NO**.

Finalmente, supongamos que en la fase adivinadora se construye un subconjunto $S \subseteq O$ tal que el costo w de S no sobrepasa la capacidad C , pero la ganancia total del subconjunto no es mayor o igual a k . En este caso: Como no sobrepasa la capacidad C de la mochila, no entra en el **if** de la línea 16. Pasa a calcular su ganancia total. Como la ganancia total no es mayor o igual a k , entonces entra en el **if** de la línea 22 y regresa **NO**.

Como podemos ver, el algoritmo siempre regresa **YES** cuando se cumple que el costo de S no sobrepasa la capacidad C de la mochila y la ganancia de S es mayor o igual a k . En cualquier otro caso, regresa **NO**. Por lo tanto el algoritmo es correcto.

1.3. Análisis de tiempo de ejecución

En la fase no determinista, para crear el subconjunto $S \subseteq O$, tenemos que recorrer con un ciclo **for** todos los elementos de O para determinar si los agregamos o no a S . Internamente en dicho ciclo se ejecuta nuestra función **nd-choice**, que consiste en lanzar una moneda. Esta operación toma tiempo $O(1)$. Como esta operación está dentro del **for** y se ejecuta n veces (donde n es el número de elementos en O), la fase no determinista toma tiempo $O(n)$.

Una vez construido el subconjunto S , pasamos a la fase de verificación: En la línea 13, iteramos sobre los elementos de S , que tiene cardinalidad $m \leq n$. Dentro del **for**, se efectúa una operación que suma los pesos w de los elementos de S , lo cual toma tiempo $O(1)$. Como se ejecuta m veces, toma tiempo $O(m)$, que en el peor caso será $O(n)$ si S contiene todos los elementos de O . El **if** de la línea 16 es una simple comparación que toma $O(1)$. El **for** de la línea 19 también toma $O(n)$ por la misma razón que el **for** de la línea 13. El **if** de la línea 22, al ser una comparación, toma tiempo $O(1)$.

En total, toda la fase verificadora toma tiempo $O(n)$.

Considerando que tanto la fase adivinadora como la fase verificadora toman tiempo $O(n)$, podemos concluir que este es un **algoritmo no determinista y polinomial**.

2. Problema del producto de subconjunto

2.1. Pseudocódigo del algoritmo

Algorithm 2 checkSubsetProduct Function

```
1: Function checkSubsetProduct( $A, t, s, k$ ):  
2: # Guessing phase  
3:  $x \leftarrow \{\}$  {Initialize empty set}  
4: for each  $i \in A$  do  
5:   result  $\leftarrow$  flip a coin  
6:   if result = heads then  
7:      $x \leftarrow x \cup \{A[i]\}$  {Add  $A[i]$  to set  $x$ }  
8:   end if  
9: end for  
10: # Verification phase  
11:  $product \leftarrow 1$  {Initialize product to 1}  
12: for each  $i \in x$  do  
13:    $product \leftarrow product \times x[i]$   
14: end for  
15: if  $product \leq t$  and  $product \geq k$  then  
16:   return YES  
17: end if  
18: return NO
```

3. Demostración y Análisis del Algoritmo

El algoritmo no determinista `checkSubsetProduct` es correcto para el Problema del Producto de Subconjunto.

Supongamos que en la fase verificadora se crea un subconjunto $x \subseteq A$ tal que el producto de todos sus elementos es mayor o igual a k .

Pueden ocurrir dos casos:

Primer caso: El producto total de todos los elementos de x es mayor que t . En este caso, no se cumplen las condiciones del `if` de la línea 15, por lo que se ejecuta la línea 18 y el algoritmo regresa **NO**.

Segundo caso: El producto total de todos los elementos de x es menor o igual a t . En este caso, se cumplen las condiciones del `if` de la línea 15, y el algoritmo regresa **YES** en la línea 16.

Ahora, supongamos que en la fase verificadora se crea un subconjunto $x \subseteq A$ tal que el producto de todos sus elementos es menor que k . En este caso:

No importa si el producto es menor o igual a t , ya que no se cumple la condición de maximización. No entra en el `if` de la línea 15, y por lo tanto el algoritmo regresa **NO** en la línea 18.

Por lo tanto, podemos concluir que el algoritmo `checkSubsetProduct` regresa **YES** cuando el producto del subconjunto x supera k pero no sobrepasa t , y regresa **NO** cuando alguna de estas condiciones no se cumple.

3.1. Análisis de tiempo de ejecución

En la fase adivinadora, se recorre el conjunto A con n elementos mediante un ciclo `for`. Dentro del ciclo `for` se ejecuta la función `nd-choice`, que consiste en lanzar una moneda y, si cae cara, agregar el elemento iterado al subconjunto x . Esta operación toma tiempo $O(1)$. Dado que esta acción se repite n veces, la complejidad de la fase adivinadora es $O(n)$.

En la fase verificadora, se utiliza un ciclo `for` que itera sobre los elementos del subconjunto x creado en la fase adivinadora, cuyo tamaño es $m \leq n$. Dentro del ciclo:

Se realizan dos comparaciones en un `if`, las cuales toman $O(1)$. Esta acción se repite m veces, resultando en una complejidad total de $O(m)$, que en el peor caso será $O(n)$ cuando x contenga todos los elementos de A .

En general, la complejidad de todo el algoritmo es $O(n)$.

Dado que tanto la fase adivinadora (no determinista) como la fase verificadora tienen complejidad $O(n)$, podemos concluir que la función `checkSubsetProduct` es un **algoritmo no determinista y polinomial**.

4. Anexo

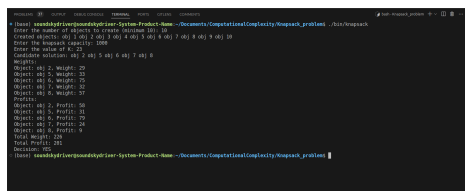
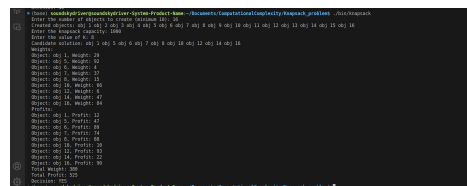
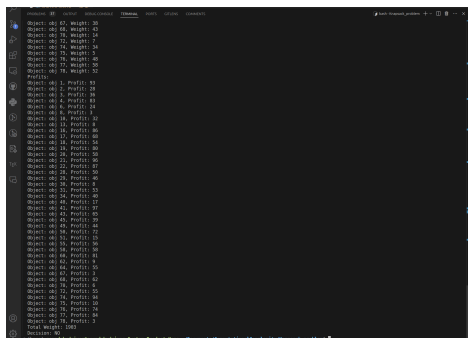
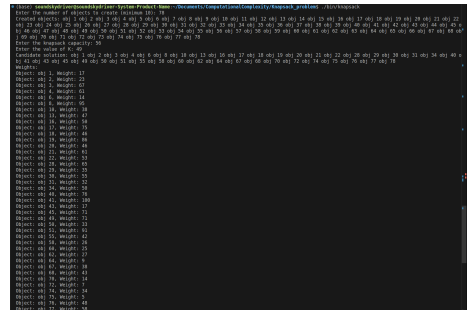
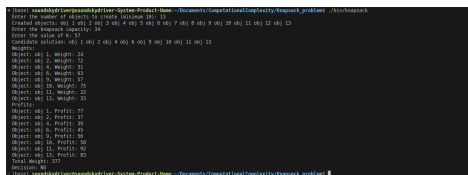


Figura 6: Captura de pantalla de 5 ejecuciones del problema de la mochila

[illegible]

Figura 7: Imagen 1

[illegible]

Figura 8: Imagen 2

[illegible]

Figura 9: Imagen 3

[illegible]

Figura 10: Imagen 4

[illegible]

Figura 11: Imagen 5

Figura 12: Captura de pantalla de 5 ejecuciones del problema del producto de subconjunto